

ROS 2 Indoor Robot Project Plan

Mission: Find Target, Report, Return Home | **Platform:** TurtleBot 3 · ROS 2 Humble · Gazebo · WSL 2 (Ubuntu 22.04) | **Duration:** ~4 weeks + Week 0 (~90–100 hours)

Primary learning goal: Understand ROS 2 and the mobile robotics software stack — how sensors, maps, localization, navigation, perception, and mission logic connect.

Main mission goal: A robot that localizes in a known environment, navigates to a given target area, finds a target with its camera, reports success, and returns home.

Table of Contents

1. Project Summary & Scope
 2. Project Description (Draft with Measurable Levels)
 3. Reflection Answers & Adjustments
 4. System Architecture
 5. Mission Flow
 6. Go / No-Go Gates
 7. Week 0 — Environment Proof
 8. Week 1 — ROS 2 Foundations + Simulation
 9. Week 2 — Space, TF, and Maps
 10. Week 3 — Autonomous Navigation
 11. Week 4 — Perception, Report, Full Mission
 12. Definition of Complete
 13. Risks & Mitigations
 14. Resources
 15. Daily Journal Template
-

1. Project Summary & Scope

One-sentence version

In simulation, build a small wheeled robot that knows where it is in a map, drives to chosen spots on its own, spots a target with a camera, reports success, and drives back home.

What the robot does (end state)

1. **Localizes** — knows where it is on a 2D map (AMCL).
2. **Navigates** — drives to a pose you give it (Nav2).
3. **Perceives** — camera finds a target (ArUco marker recommended).
4. **Reports** — publishes a message (e.g. `target_found`).
5. **Returns home** — Nav2 drives back to the home pose.

Mission story

“Go to the search area, find the package (marker), report success, return to base.”

In scope

- Localization on a 2D occupancy grid map
- Autonomous navigation via Nav2
- Camera-based target detection (start with ArUco)
- Mission state machine (Python node you write)
- Map: pre-made first; building map yourself (SLAM) as milestone or stretch

Out of scope (month 1)

- Robot arm / tapping or touching the target (stock TurtleBot 3 has no manipulator)
- Training a large deep-learning model from scratch
- Multi-robot coordination
- Custom robot URDF from scratch in week 1

Time budget

Item	Value
Total hours	~90–100

Daily focus	~3–3.5 hours
Duration	~4 weeks + Week 0

Priority rule: If time runs short, protect navigation + return home before fancy perception.

2. Project Description — Measurable Learning Levels

Learning is the underlying goal, but these checkpoints make progress tangible.

Level 1 — “I can run and observe the stack” (Week 1)

Done when:

- Robot spawns in simulation
- You can drive it with the keyboard (teleop)
- You can see wheel odometry (`/odom`), laser scan (`/scan`), and velocity commands (`/cmd_vel`)
- You can draw which **nodes** talk to which **topics**

You can explain: What is a node? What is a topic? What does the robot publish vs. what you send to move it?

Level 2 — “I understand space and mapping” (Week 2)

Done when:

- You can explain the **TF tree** (coordinate frames: map, odom, base_link, sensor frames)
- You have a map file loaded in RViz (built via SLAM or pre-made)
- Laser scan aligns with walls on the map

Terms: **TF** = system tracking where each part of the robot is relative to the world. **SLAM** = building a map while estimating where you are.

You can explain: Mapping vs. localization? Why more than one coordinate frame?

Level 3 — “I can navigate on purpose” (Week 3)

Done when:

- Robot localizes on saved map (AMCL)
- You send a goal pose; Nav2 drives there without keyboard

- Robot completes 3 goals in a row ($A \rightarrow B \rightarrow C$) with at most minor recovery events

Terms: **AMCL** = localization that matches laser scans to the map. **Nav2** = standard ROS 2 navigation (planner + obstacle avoidance + controller).

You can explain: What Nav2 does vs. what your mission code does?

Level 4 — “Perception hooks into mission logic” (Week 4)

Done when:

- Camera detects target (ArUco or colored object)
- Mission node runs: HOME $\rightarrow$ GO_TO_SEARCH $\rightarrow$ DETECT $\rightarrow$ REPORT $\rightarrow$ GO_HOME $\rightarrow$ DONE
- On detection, robot publishes `target_found` and returns home via Nav2

Stretch: Navigate closer to target before returning. No tapping on stock TurtleBot 3.

Weekly measurable artifacts

Week	Tangible artifact	Knowledge check
1	Topic list / teleop working	Explain pub/sub and 3 key topics
2	Saved or loaded map file	Draw TF chain map $\rightarrow$ odom $\rightarrow$ base_link
3	Nav2 reaches 3 poses	Explain costmap vs planner vs controller
4	Mission log: detect + return	Explain mission vs navigation split

3. Reflection Answers & Plan Adjustments

Your stated priorities

- **Learning over demo polish** — correct for month 1; use checkpoints above to avoid endless tutorial hopping.
- **Mapping / localization / navigation understanding** — your mental model is sound; refinements below.
- **Mission:** Find target and report (not tap). Signal/message is the right interaction for TurtleBot 3.
- **Environment:** Windows 11 + WSL 2 + Ubuntu 22.04 (fresh install). Week 0 validates RViz/Gazebo before core content.

Mapping vs localization vs navigation (refined)

Layer	What it does
Mapping	Build a representation of the environment (usually 2D occupancy grid from lidar). Often done once, then map is saved and reused.
Localization	Estimate where the robot is on that map (pose: x, y, yaw). AMCL compares live lidar to the map. Separate from mapping — you can localize on a map you did not build.
Navigation	Given a goal pose and current pose, compute a collision-free path and drive there. Nav2 handles this. Navigation does not decide <i>which</i> goal — mission logic does.
Mission logic	Your code: “Go to search area, look for target, report, go home.”

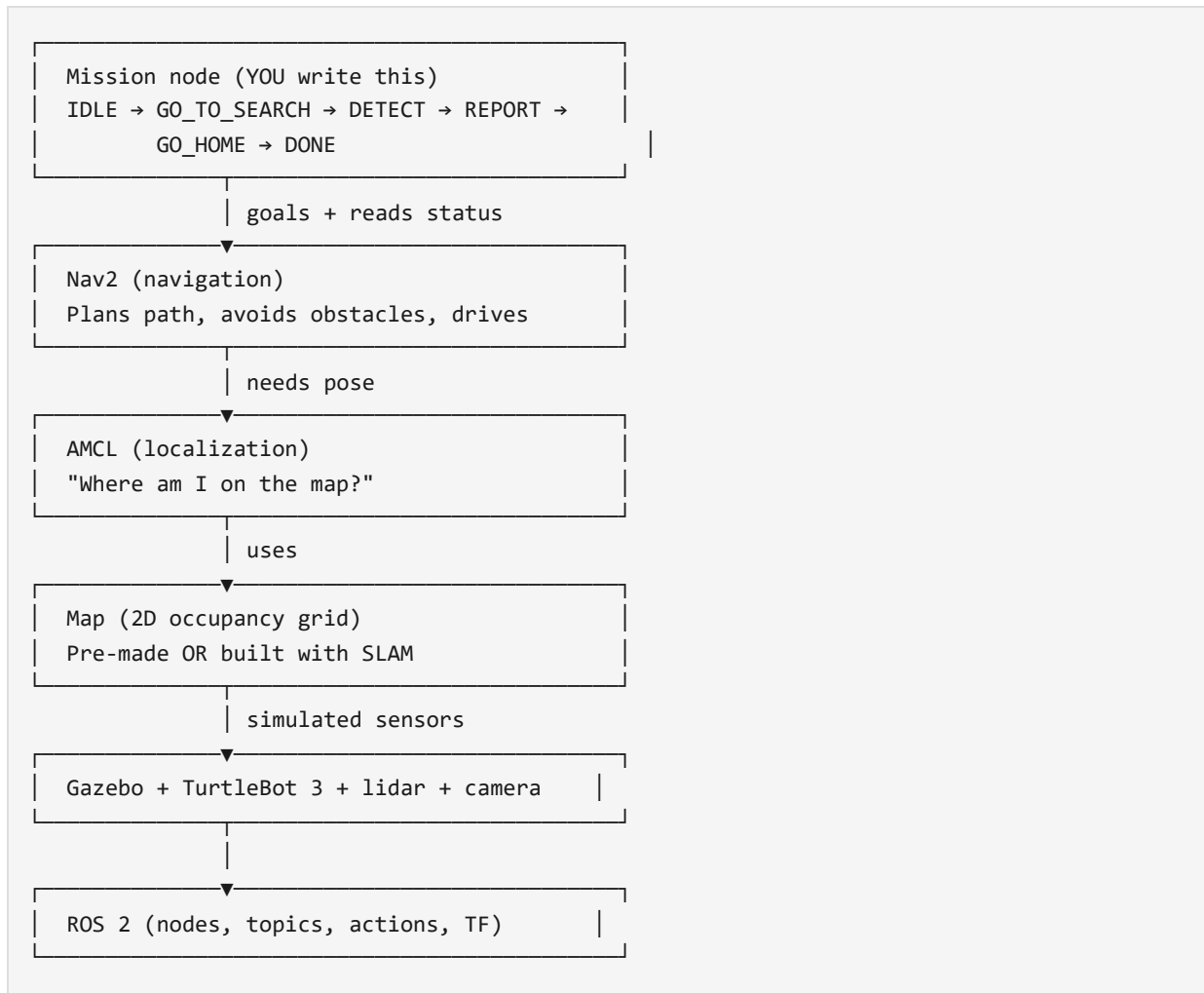
Why “tap the target” is out of scope

Standard TurtleBot 3 (Burger/Waffle) has differential drive, lidar, optional camera — **no arm or gripper**. Tapping requires a manipulator (different project). Alternatives: publish `target_found` topic; optionally navigate closer to detected pose.

WSL 2 / RViz note

Windows 11 WSLg should show Linux GUI apps without extra X server. Week 0 proves `rviz2` and simulation before SLAM/Nav2. If graphics stay painful, consider dual-boot Ubuntu.

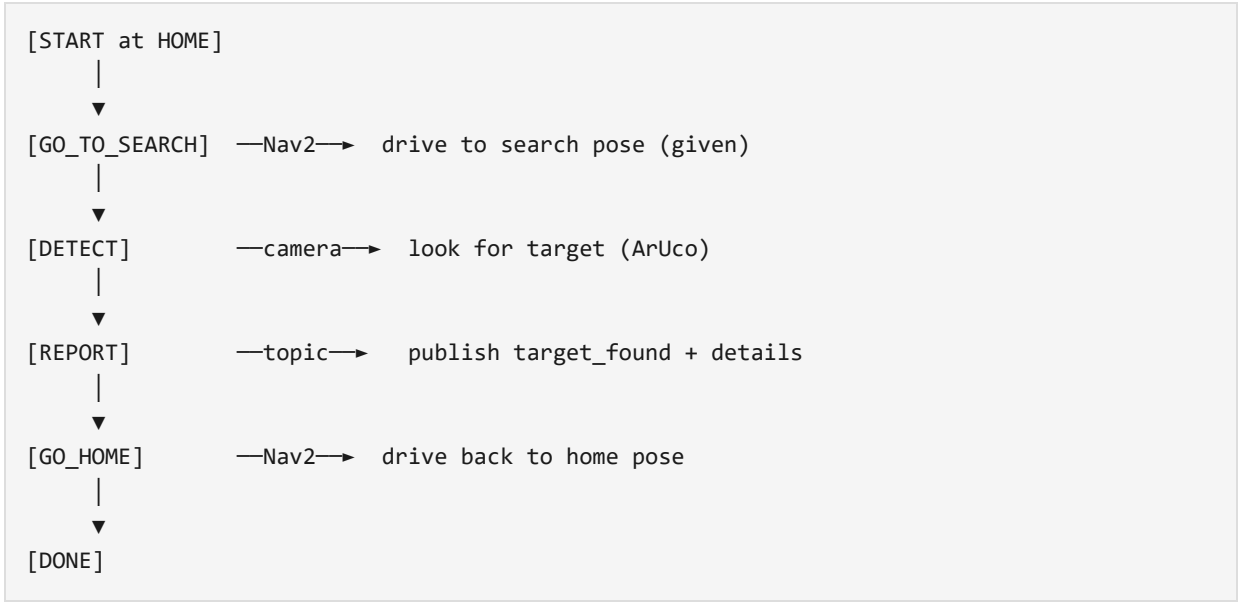
4. System Architecture



Key terms

- **Node** — one running program in ROS (navigation, camera, your mission code).
 - **Topic** — named message stream (e.g. `/scan`, `/camera/image_raw`).
 - **Action** — long-running task with feedback; Nav2 uses `NavigateToPose`.
 - **TF** — transforms between coordinate frames (map, odom, base_link, camera).
 - **URDF** — robot description file (sizes, wheel placement, sensors).
 - **Pose** — position + heading: x, y, yaw.
-

5. Mission Flow



Mission node state spec (Week 4)

State	Action
IDLE	Wait for start (service, topic, or timer)
GO_TO_SEARCH	Call Nav2 to search pose; on success → DETECT
DETECT	Wait N seconds; if detection → REPORT; else retry or fail gracefully
REPORT	Publish report message → GO_HOME
GO_HOME	Nav2 to home pose → DONE
DONE	Publish mission complete

Record **home pose** and **search pose** in docs/poses.md once the map works.

6. Go / No-Go Gates

Gate	Pass before
Go	Week 0: ROS talker/listener + RViz opens
G1	Week 1: teleop + see /scan and /odom
G2	Week 2: understand TF; map file loaded in RViz

G3	Week 3: Nav2 reaches one goal reliably
G4	Week 4: full mission once (detect + report + return)

7. Week 0 — Environment Proof (~8–12 hours)

Objective

Prove WSL 2 + Ubuntu 22.04 + ROS 2 Humble + GUI works before robotics content.

Deliverables

- `xeyes` or similar GUI works (WSLg)
- ROS 2 Humble installed; talker + listener communicate
- `rviz2` opens
- TurtleBot 3 simulation launches; laser visible in RViz
- Keyboard teleop moves the robot

Day-by-day

Day	Tasks	Hours
0	WSL update; <code>lsb_release -a</code> ; GUI test (<code>xeyes</code>)	1–2
1	Install ROS 2 Humble; talker/listener	2–3
2	Install RViz; TurtleBot 3 + sim packages	2–3
3	Launch empty world + teleop; fix display/sim issues	2–4

Smoke-test phases

Phase A — WSL basics: Ubuntu terminal; `sudo apt update && sudo apt upgrade`; confirm 22.04.

Phase B — GUI: `sudo apt install x11-apps -y`; run `xeyes`. Window should appear on Win 11.

Phase C — ROS 2: Install Humble; `source /opt/ros/humble/setup.bash`; run talker + listener in two terminals.

Phase D — RViz: `ros-humble-rviz2`; run `rviz2`.

Phase E — Simulation: Launch TurtleBot 3 world; RViz LaserScan on `/scan`; teleop moves robot.

Week 0 decision

Result	Next step
All pass	Start Week 1
GUI OK, sim flaky	Debug sim only; don't install Nav2 yet
RViz/Gazebo unusable	Consider dual-boot Ubuntu before Week 3

Common mistakes

- Forgetting source `/opt/ros/humble/setup.bash` in each new terminal
 - Mixing ROS 1 and ROS 2 tutorials
 - Debugging Nav2 when the real problem is WSL graphics
-

8. Week 1 — ROS 2 Foundations + Simulation (~20–24 hours)

Objective

Operate the simulated robot manually and read the data flow.

Concepts

Nodes, topics, pub/sub, launch files, parameters, `/cmd_vel`, `/scan`, `/odom`, basic URDF.

Deliverables

- G1: Teleop works
- List nodes and topics while sim runs
- Diagram: teleop → `cmd_vel` → robot → `scan/odom`
- Optional: create empty Python ROS 2 package in workspace

Tasks

1. Run TurtleBot 3 sim + teleop
2. `ros2 topic list`, `ros2 topic echo /scan --once`, `ros2 node info`
3. Read one launch file — what gets started?

4. RViz: LaserScan, Odometry
5. Create mission_robot package shell in src/

Suggested repo structure

```
ros_robot/  
├── docs/  
│   ├── system_graph.md  
│   └── poses.md  
├── src/  
│   └── mission_robot/  
│       ├── package.xml  
│       ├── setup.py  
│       └── mission_robot/  
└── README.md
```

Reflection

- Who publishes `/cmd_vel` when you are not teleoperating?
 - What is the difference between odometry and a map?
-

9. Week 2 — Space, TF, and Maps (~22–26 hours)

Objective

Understand where the robot is in space; load or build a map.

Concepts

TF/tf2, frames (map, odom, base_link, base_scan, camera), SLAM Toolbox, map save/load, AMCL preview.

Deliverables

- G2: Map file (`.yaml` + `.pgm`) — SLAM or pre-made
- TF diagram: map → odom → base_link → sensors
- RViz: map + laser aligned
- Write-up: mapping vs localization in your words

Two paths

Path A (faster): Use default TurtleBot 3 navigation demo map; focus on TF + RViz. Defer self-built map to stretch.

Path B (full): Teleop + SLAM; save map with `map_saver`; reload in new session.

Tasks

1. `ros2 run tf2_tools view_frames`
2. RViz: Fixed Frame = map; add Map, LaserScan, TF
3. SLAM + teleop OR load existing map
4. Launch AMCL; 2D Pose Estimate in RViz

Common mistakes

- Wrong fixed frame in RViz
 - Saving map before exploring full space
 - Confusing odom drift with map localization error
-

10. Week 3 — Autonomous Navigation (~24–28 hours)

Objective

Robot drives to given poses without keyboard — backbone of the mission.

Concepts

Nav2, global/local costmaps, planners, controllers, recovery behaviors, `NavigateToPose` action.

Deliverables

- G3: Single Nav2 goal in RViz succeeds
- Three poses in sequence ($A \rightarrow B \rightarrow C$)
- Home + search poses in `docs/poses.md`
- Python script: `NavigateToPose` to home and search area
- Chain: search pose $\rightarrow$ wait $\rightarrow$ home (no camera yet)

Tasks

1. Launch TurtleBot 3 navigation bringup (map + AMCL + Nav2)
2. Set initial pose; send goal; watch plans in RViz
3. Skim Nav2 params (robot radius, max speed)
4. Practice recovery behaviors
5. Python NavigateToPose node
6. Go to search → go home without perception

Risk: Highest-risk week. Budget 2 extra sessions for Nav2 debugging.

Reflection

- Global planner vs local controller?
- Should your mission node publish `/cmd_vel`? Why or why not?

11. Week 4 — Perception, Report, Full Mission (~20–24 hours)

Objective

Find target → report → return home under mission control.

Concepts

Image topics, ArUco detection, custom/std messages, finite state machine, Nav2 + perception integration.

Deliverables

- G4: Full state machine runs end-to-end
- Publish `/mission/target_found` (or similar) on detection
- Return home after report
- Complete `docs/system_graph.md`
- Explain stack top-to-bottom (~5 min out loud)

Perception order (easiest first)

1. ArUco marker in Gazebo (recommended)
2. Colored blob (OpenCV)
3. Small ML detector (stretch only)

Tasks

1. View `/camera/image_raw` (`rqt_image_view` or `RViz`)
2. Detection node: `image_in` → `target_detected` out
3. Place marker in Gazebo at search area
4. Mission FSM with timeouts in `DETECT`
5. Log to `/mission/status`
6. End-to-end test 3×; document failure modes

Common mistakes

- Blocking node in a tight loop (use timers / async callbacks)

- No timeout in DETECT
- Changing too many Nav2 YAML params at once

Optional challenges

- Build own map with SLAM; run full mission
- Navigate closer using estimated object pose
- Rosbag replay for perception debugging

12. Definition of “Project Complete”

1. **Localization:** AMCL on 2D map; laser aligns with map in RViz.
2. **Navigation:** Given target pose, Nav2 drives there reliably.
3. **Mission:** Search → detect → report on topic → home.
4. **Understanding:** System graph + explain each layer without notes.
5. **Mapping (stretch):** You built and saved the map yourself.

Checkpoint rubric

Score	Meaning
Green	Deliverables done + reflection questions answered
Yellow	Partial; carry one item forward, no new scope
Red	Stuck >2 sessions → simplify (e.g. default map, stub detection)

If only ~80 hours

1. Week 0 + Week 1 (full)
2. Week 2 Path A (existing map + TF)
3. Week 3 (full) — non-negotiable
4. Week 4 minimal: manual `target_found` + return home, then real camera

13. Risks & Mitigations

Risk	Mitigation
------	------------

WSL graphics issues	Week 0 gate; WSLg on Win 11; dual-boot fallback
Nav2 config complexity	Stock TurtleBot 3 nav launch unmodified first
Week 3 eats Week 4	Pre-define poses; stub DETECT with manual topic
Perception unreliable	ArUco only; DETECT timeout; save failure images
Scope creep (tap object)	Out of scope; report topic = success

14. Resources (official-first)

Topic	Link
ROS 2 Humble install	https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debians.html
ROS 2 tutorials	https://docs.ros.org/en/humble/Tutorials.html
TF2 tutorials	https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Tf2-Main.html
TurtleBot 3 e-manual	https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/
Nav2	https://navigation.ros.org/
SLAM Toolbox	https://github.com/SteveMacenski/slam_toolbox
ROS 2 Actions (Python)	https://docs.ros.org/en/humble/Tutorials/Intermediate/Writing-an-Action-Server-Client/Py.html

15. Daily Journal Template

Save in `docs/system_graph.md` (~10 min/day):

```
Date:
Running launch files:
Nodes (list):
Topics I care about:
TF chain:
What I changed today:
What broke:
One sentence: what does the system do now?
```


Document generated for ROS robot learning project — Find, Report, Return. Platform: TurtleBot 3, ROS 2 Humble, Gazebo, WSL 2 Ubuntu 22.04.